

AWS Architecture Project Report

Serverless Observability & Incident Response via EMF

Prepared for: AWS Certified DevOps Engineer - Professional (DOP-C02)

June 2026

Executive Summary

This report documents the architectural design, deployment, and chaos engineering validation of a serverless observability pipeline. The project utilizes AWS Lambda, Amazon CloudWatch Embedded Metric Format (EMF), CloudWatch Alarms, and Amazon SNS. The primary objective is to implement decoupled, asynchronous metric generation to monitor application-level health (Latency, Errors, Throttles) while strictly controlling AWS API costs and minimizing execution latency.

1 Architectural Blueprint

The solution avoids synchronous AWS SDK calls (`PutMetricData`) by leveraging standard output logs to generate custom metrics.

- **Compute Layer:** AWS Lambda (Python 3.12) generating simulated traffic and business states.
- **Ingestion Layer:** Lambda `stdout` stream implicitly writes to Amazon CloudWatch Logs.
- **Metric Extraction:** CloudWatch asynchronously parses logs containing the `_aws` directive to generate Custom Metrics.
- **Evaluation & Alerting:** CloudWatch Alarms evaluate the `AppErrors` and `AppThrottles` metrics, triggering an Amazon SNS Topic for email notification.

2 Infrastructure as Code (IaC) Highlights

The deployment was entirely automated using AWS CloudFormation, reflecting Domain 1 (SDLC Automation) of the DOP-C02 exam.

Critical Security Configurations

A common deployment failure occurs when connecting CloudWatch Alarms to SNS Topics. CloudWatch Alarms do not use standard IAM Execution Roles to publish messages. Instead, the architecture requires an explicit **Resource-Based Policy** attached to the SNS Topic, granting the `cloudwatch.amazonaws.com` service principal the `sns:Publish` action.

3 Chaos Engineering & Failure Analysis

During Step 3 of the sprint, controlled failures were introduced to validate the system's resilience and dependencies.

Injected Failure	System Behavior	Architectural Resolution
Removed <code>logs:PutLogEvents</code> from IAM Role	Logs fail to write. EMF parsing stops. Custom metrics flatline instantly.	Restore <code>AWSLambdaBasicExecutionRole</code> . EMF is strictly dependent on standard logging permissions.
Lambda OOM Crash / Hard Timeout	Execution terminates before <code>print(emf_payload)</code> executes. Custom metric is lost.	Utilize standard AWS Platform Metrics (<code>AWS/Lambda Errors</code>) to catch hard container failures.
High Concurrency Burst (50 concurrent invocations)	AWS API returns synchronous 429 errors. Lambda does not execute.	Ensure <code>AppThrottles</code> is mapped. Use SQS queues upstream if traffic shaping is required.

Table 1: System Resilience and Failure Modes

4 DOP-C02 Exam Mapping

This project heavily aligns with the AWS Certified DevOps Engineer - Professional certification domains:

Domain 4: Policies and Standards Automation

- **High-Cardinality Dimension Trap:** A classic exam scenario involves massive CloudWatch billing spikes. The architectural safeguard is to log high-cardinality data (e.g., `RequestId`, `UserId`) as standard JSON properties within the EMF payload, *never* inside the `Dimensions` array. This allows querying via Logs Insights without generating thousands of billable custom metrics.

Domain 5: Incident and Event Response

- **Silent Alarm Failures:** If an alarm transitions to the `ALARM` state but no notification is received, the issue is typically either an unconfirmed SNS email subscription or a missing Resource-Based Policy on the SNS topic.

Domain 6: High Availability, Fault Tolerance, and DR

- **Cost & Performance Optimization:** Transitioning from synchronous SDK calls to asynchronous EMF logging significantly reduces Lambda execution time (reducing compute costs) and eliminates the risk of `PutMetricData` API throttling under high load.

5 Conclusion

The deployed architecture represents the industry "Golden Standard" for serverless telemetry. By decoupling metric generation from the execution path and separating application logic failures (EMF) from infrastructure failures (Platform Metrics), the system achieves deep observability with minimal cost and operational overhead.